

P6: Working with Multiples

Name: Alston Liow Guan You

Student ID: 37440985

Date: 16 August 2026

Overview

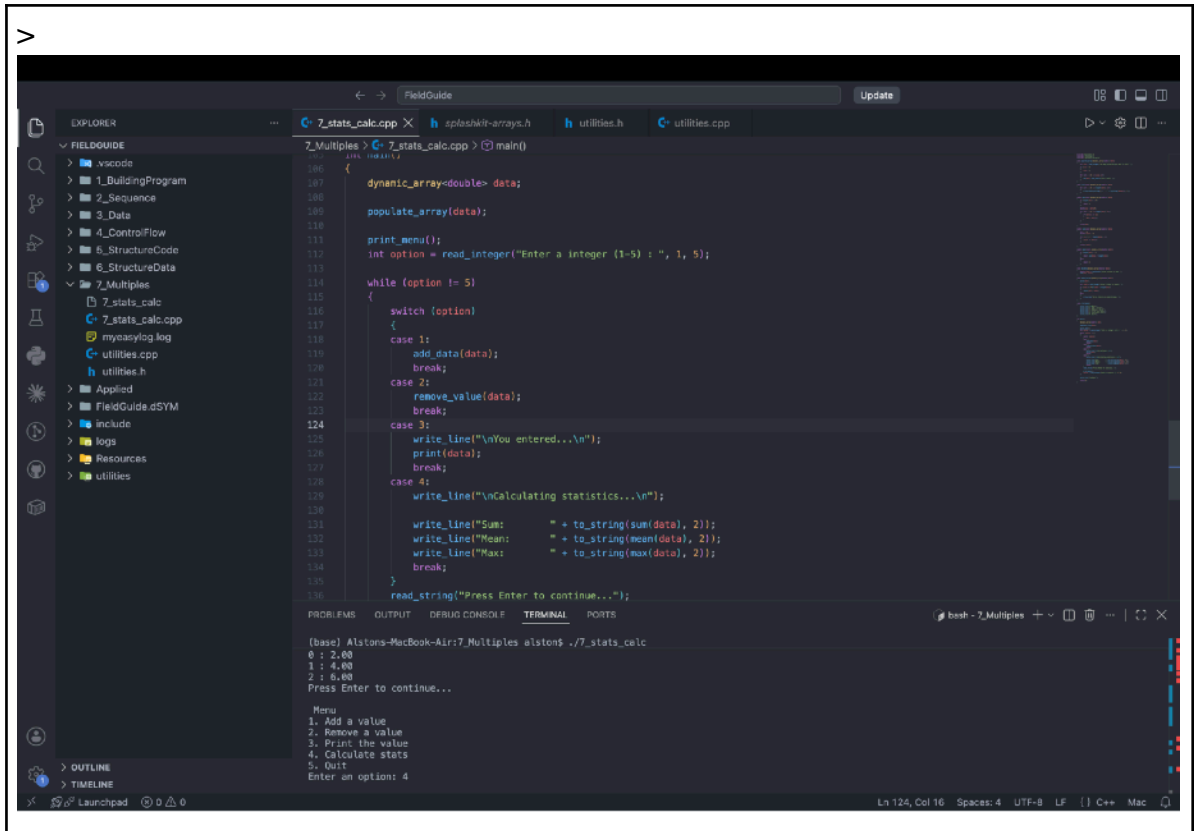
This task is an opportunity to explore and use language mechanisms to allow you to easily work with multiple values, which can grow and shrink in number as the program executes.

Tip: Systems thinking

Work to develop your ability to see how this comes together as a system. The parts are not independent of each other. Understanding each part in isolation is a good start. Make sure to follow this up by seriously focusing on seeing how the parts come together and work as a system. When you can see this, you can start to think about how to build and manipulate these systems, getting them to achieve what you want.

Do the following:

- Everything you need is in the Handling Multiples chapter of our [Programmer's Field Guide](#). The [Intro Tours](#) walk through the concepts, and should give you a good start.
- Add a screenshot image showing you were able to create and work with arrays of simple values - using a dynamic array, using interaction, for performing actions like searching, expanding, removing, and accumulating values. You can use the [statistics calculator tour](#) to achieve this (multiple pages).



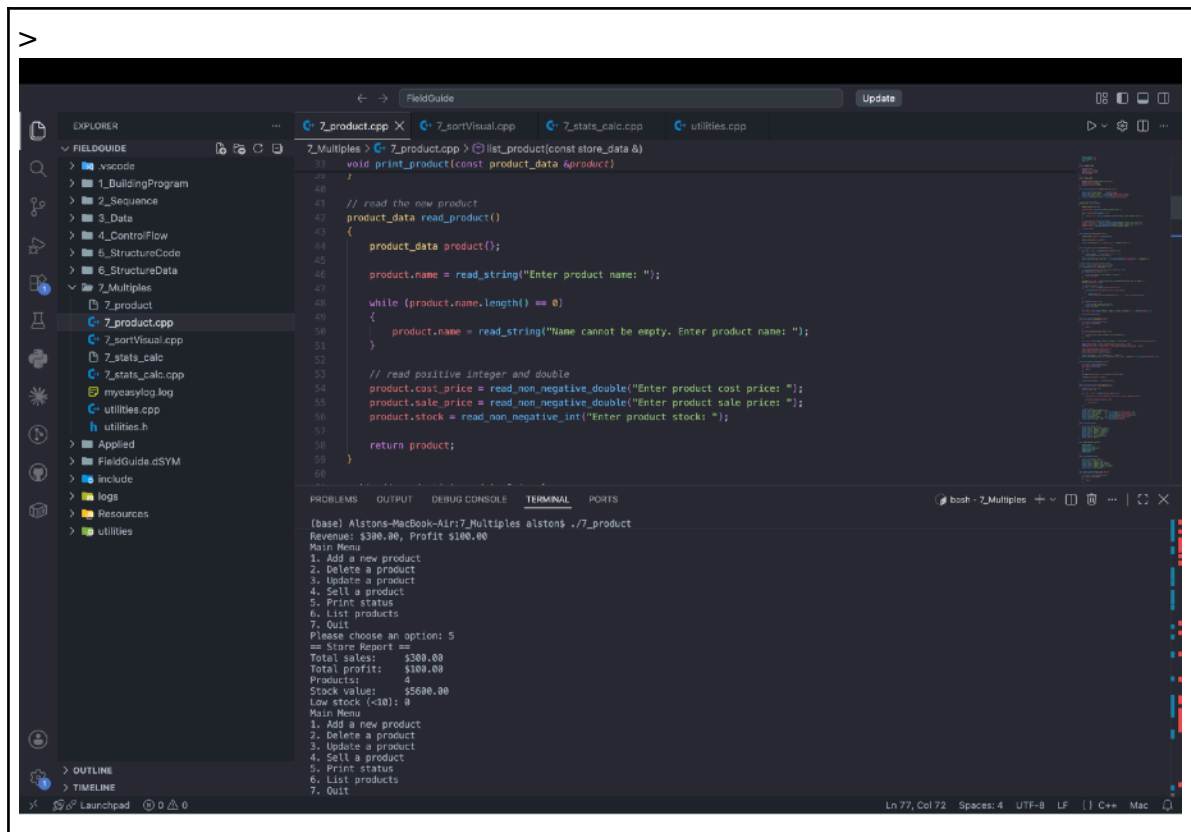
```
>
FieldGuide
7_stats_calc.cpp
106 {
107     dynamic_array<double> data;
108     populate_array(data);
109
110     print_menu();
111     int option = read_integer("Enter a integer (1-5) : ", 1, 5);
112
113     while (option != 5)
114     {
115         switch (option)
116         {
117             case 1:
118                 add_data(data);
119                 break;
120             case 2:
121                 remove_value(data);
122                 break;
123             case 3:
124                 write_line("\nYou entered...\n");
125                 print(data);
126                 break;
127             case 4:
128                 write_line("\nCalculating statistics...\n");
129
130                 write_line("Sum:      " + to_string(sum(data), 2));
131                 write_line("Mean:    " + to_string(mean(data), 2));
132                 write_line("Max:     " + to_string(max(data), 2));
133                 break;
134         }
135         read_string("Press Enter to continue...");
136     }
137 }
```

Terminal Output:

```
(base) Alstons-MacBook-Air:7_Multiples alston$ ./7_stats_calc
0 : 2.00
1 : 4.00
2 : 6.00
Press Enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculate stats
5. Quit
Enter an option: 4
```

- Add a screenshot image showing you were able to create and use arrays of your own custom types (structs). You can use the [product database tour](#) to achieve this (multiple pages).

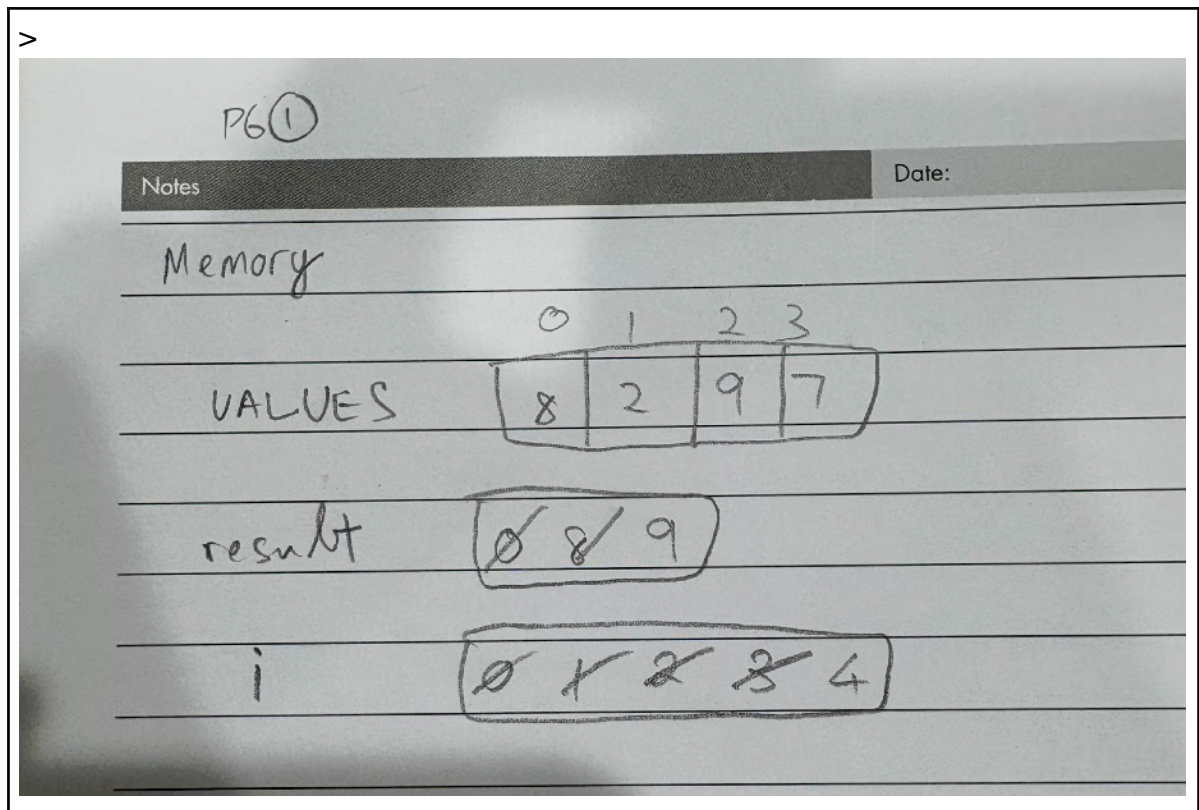


- Add a photo showing you can use our [hand execution process](#) to illustrate how code operates on arrays using the following source code.

```
const double VALUES[4] = { 8, 2, 9, 7 };
const int SIZE = 4;

double result = 0;

// Check the rest of the values
for (int i = 0; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```



- Consider the code as a system - what is it aiming to achieve? It does aim to achieve something meaningful - what is that? Can you see any issues with the way this is working in relation to what it is likely aiming to achieve?

> It is aiming to achieve to find the largest number in the arrays which is positive number.

- Repeat the hand execution for the following source code.

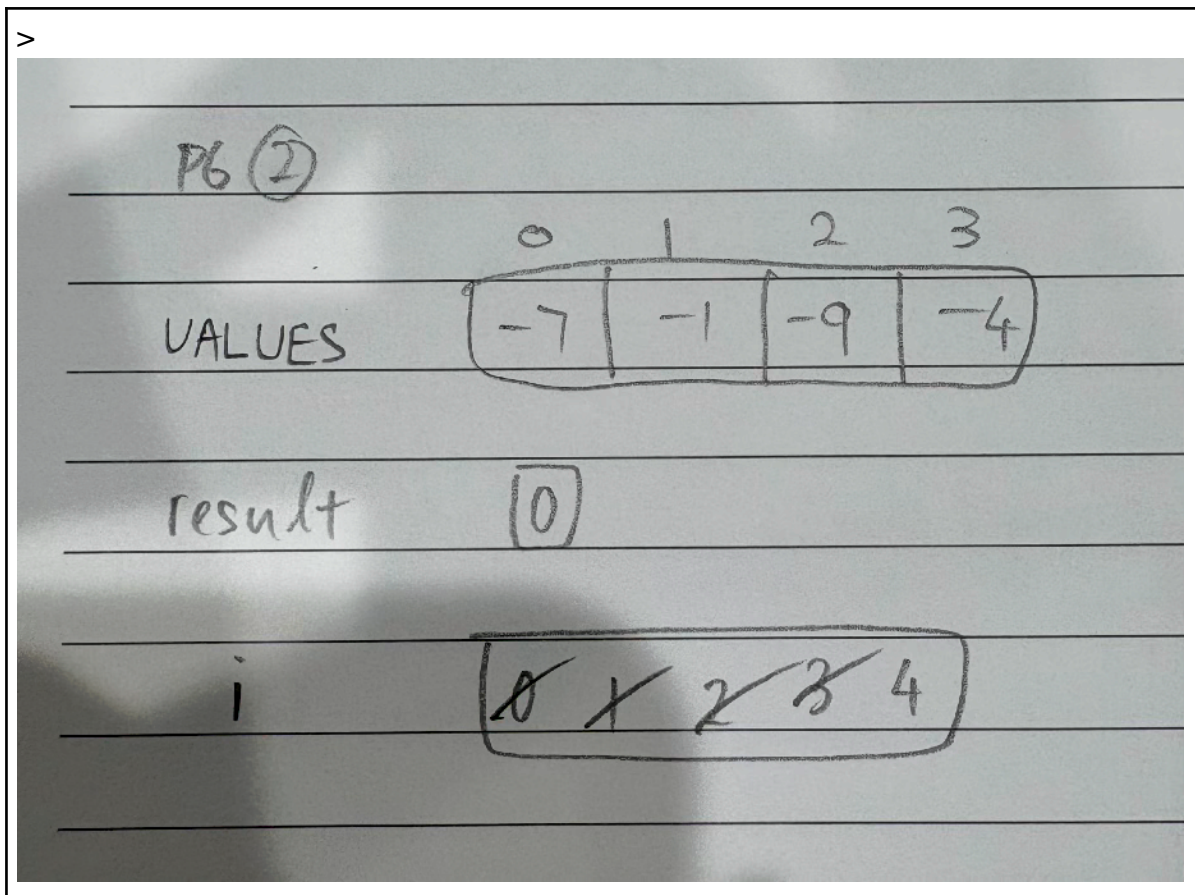
```
const double VALUES[4] = { -7, -1, -9, -4 };
const int SIZE = 4;
```

```
double result = 0;
```

```
// Check the rest of the values
```

```
for (int i = 0; i < SIZE; i++)
```

```
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```



- Repeat the hand execution for the following source code - which made two small changes to the logic!

```
const double VALUES[4] = { -7, -1, -9, -4 };  
const int SIZE = 4;
```

```
double result = VALUES[0];
```

```
// Check the rest of the values  
for (int i = 1; i < SIZE; i++)  
{  
    if (VALUES[i] > result)  
    {  
        result = VALUES[i];  
    }  
}
```

>

P6 (3)

	0	1	2	3
VALUES	-7	-1	-9	-4
result	-7	-1		
i	1	2	3	4

- How did the change impact the result? What does this say about the way these systems are put together? Consider how this impacts cases where others (including AI) are writing the code for you.

> Before changing, the result in the VALUE compare its with 0, after changing, it compare with VALUE[0], which inside the VALUE, so the changing make it can find the largest number inside the Arrays.

Tip: Focus and attention to details

Computing capabilities require you to be able to be meticulous, pay attention to details, and be able to focus on things for a period of time. These hand execution tasks will exercise these attributes. Make sure you can do this in a distraction free environment, and that you force yourself to focus on just this activity. It is a little mindless, you are wanting to show you *know* what the computer does. But you need to be able to step through things at this level to really see what the computer is doing - it is unintelligent, so we are relying on your intelligence to make it do the right thing!

- Now you need to demonstrate you can perform hand execution on more complex code. You should be able to hand-execute code without understanding the algorithm itself - the computer is unintelligent and it can follow the steps.

```
const int MAX_SIZE = 5;
```

```
double data[MAX_SIZE] = {5, 3, 2};
int size = 3;

double temp;
for (int i = 0; i < MAX_SIZE - 1; i++)
{
    for (int j = 0; j < size - i - 1; j++)
    {
        if (data[j] > data[j + 1])
        {
            temp = data[j];
            data[j] = data[j + 1];
            data[j + 1] = temp;
        }
    }
}
```

PS (4)

	0	1	2	3	4
data =	8	8	2		
	8	8	5		
	2	2			
		3			

temp 8 8 3

i 8 1 2 3 4

(i=0) j 8 2

(i=1) j 8 1

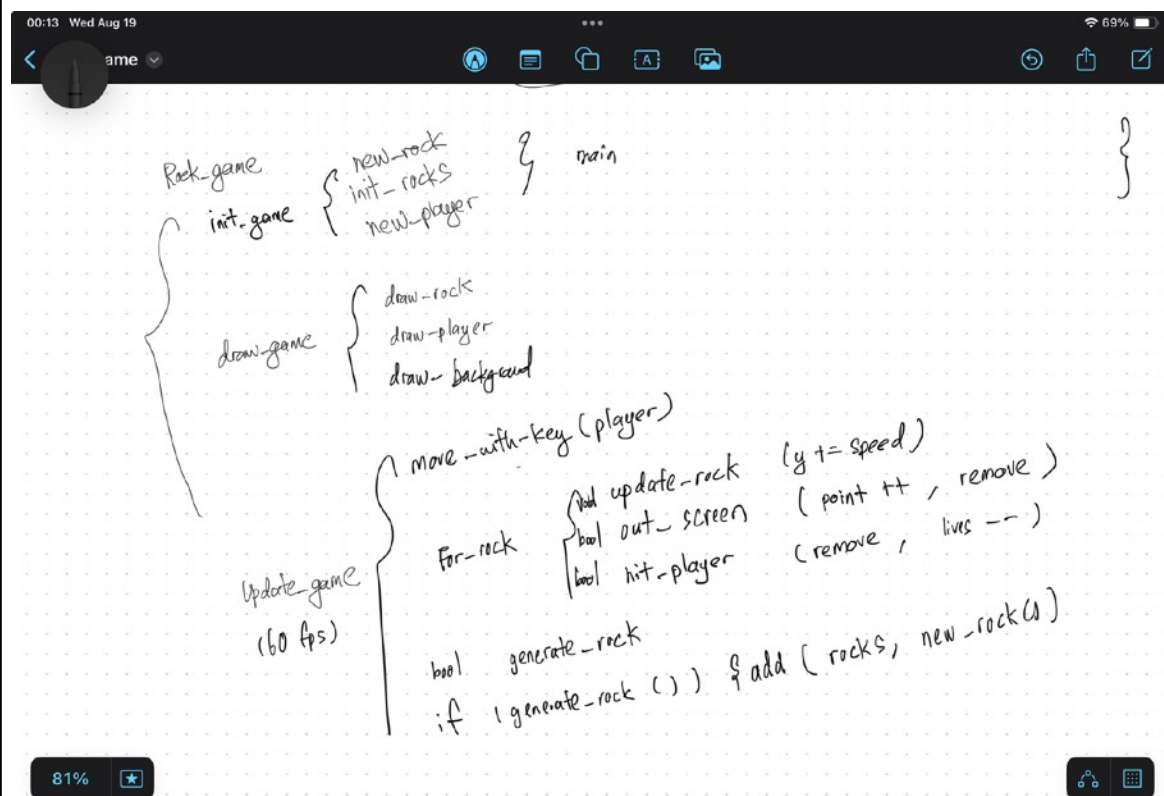
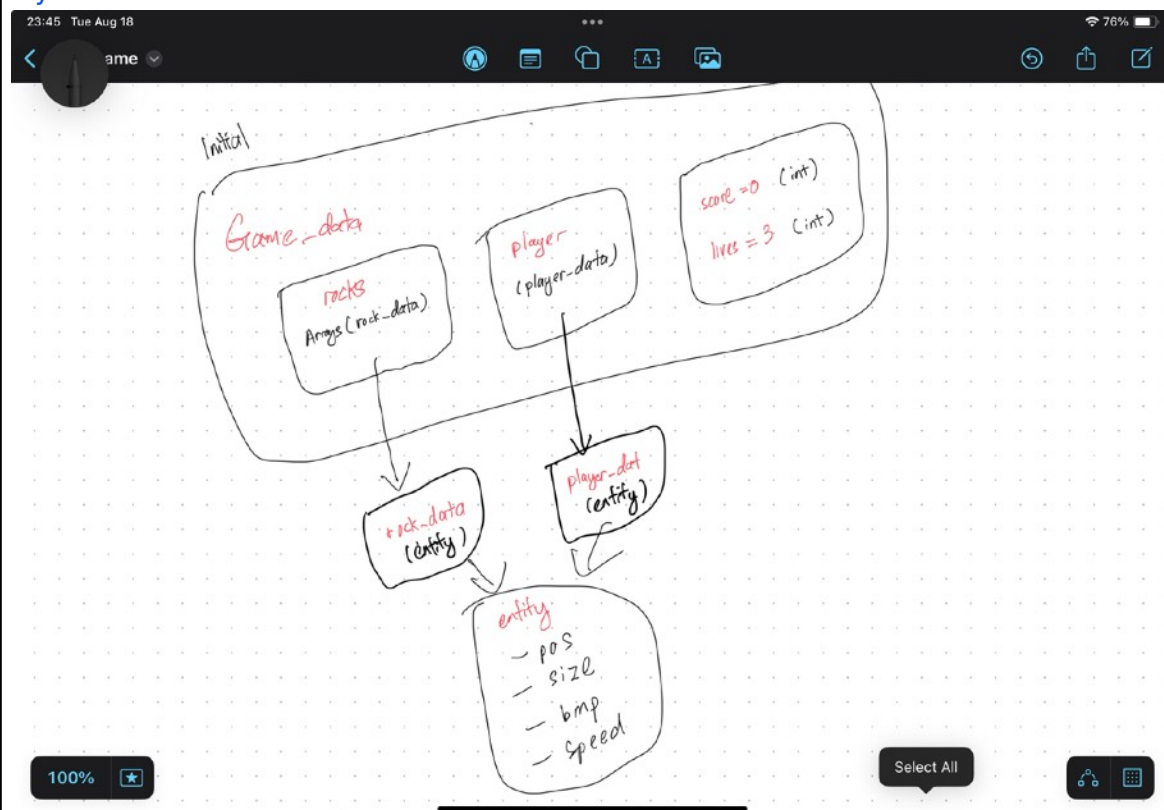
(i=2) j 0

(i=3) j 0

>

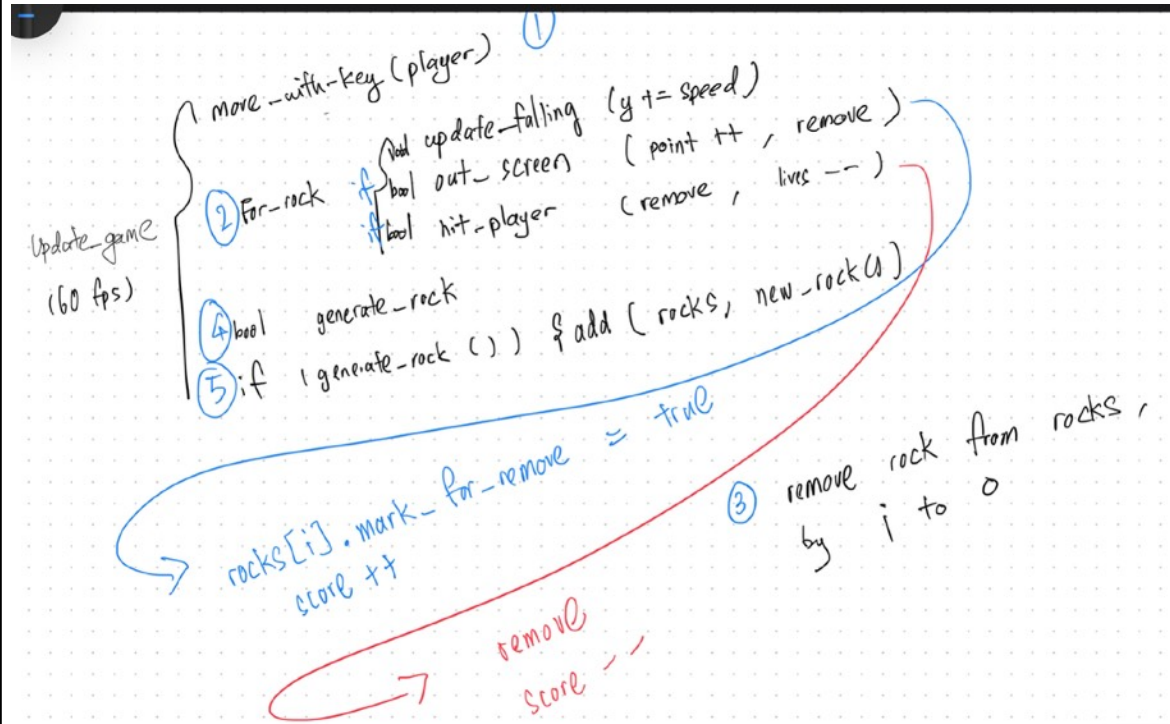
- Complete one of the [Test Your Capabilities](#) tasks from the chapter to see that you can now put these tools together to achieve something yourself.

> I choose the rock game. First I get some reference at the fly catch and map explorer, and plan about the structure of data and function I need to use about this game, this is my initial structure.

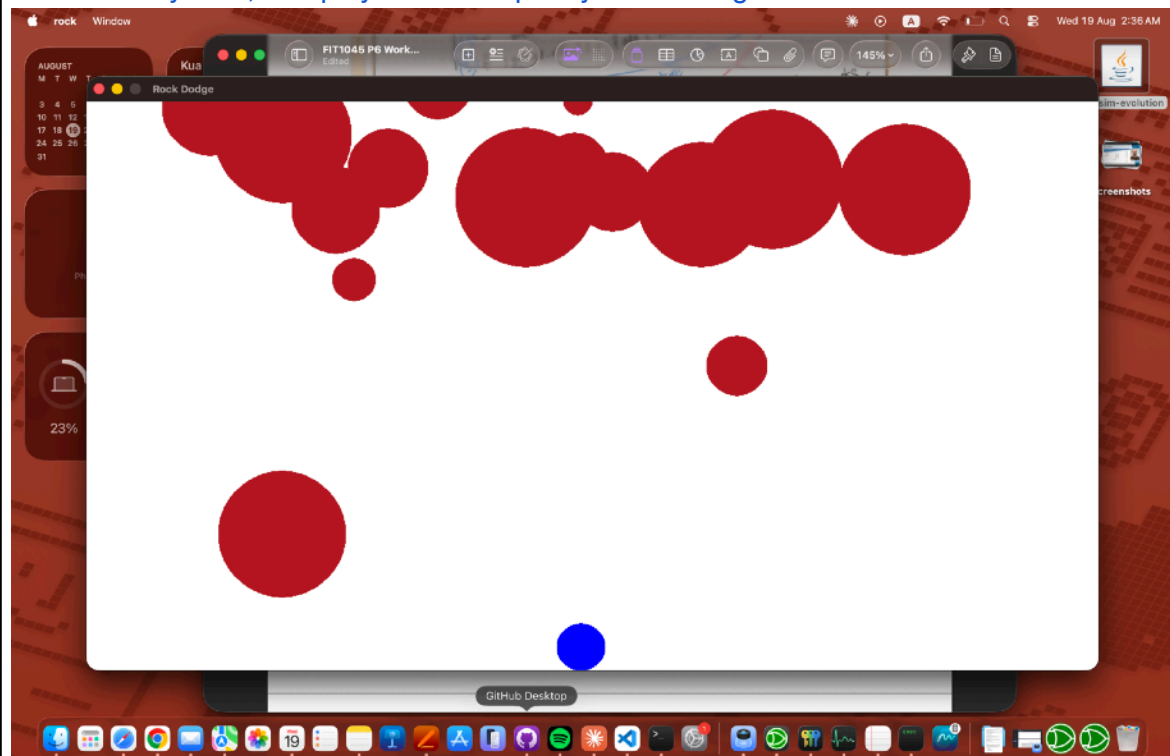


I write all the function step by step until need to combine all update function in to `update_game()`, I get stuck here. I have some idea to add something in this game, for example is typed_down(SPACE_KEY) to jump , and I want to add a horizontal thing like the rock. But I just put beside and do the main thing first.

I analyse the `update_game()` like this



First time I complete this game I forget to add a `const int MAX_ROCK_COUNT = 100`, so it is too many rock, and player will die quickly. Then the game become like this

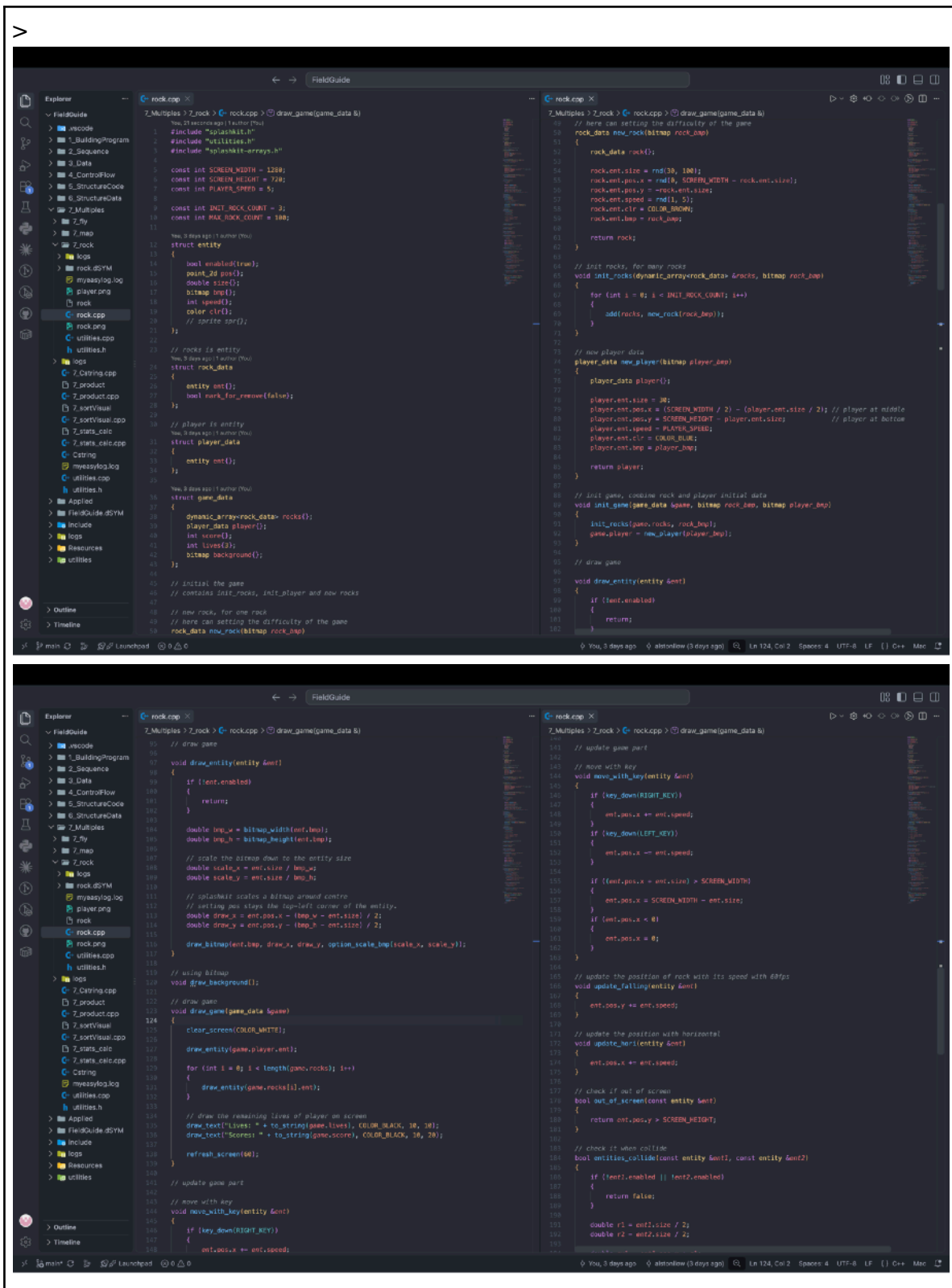


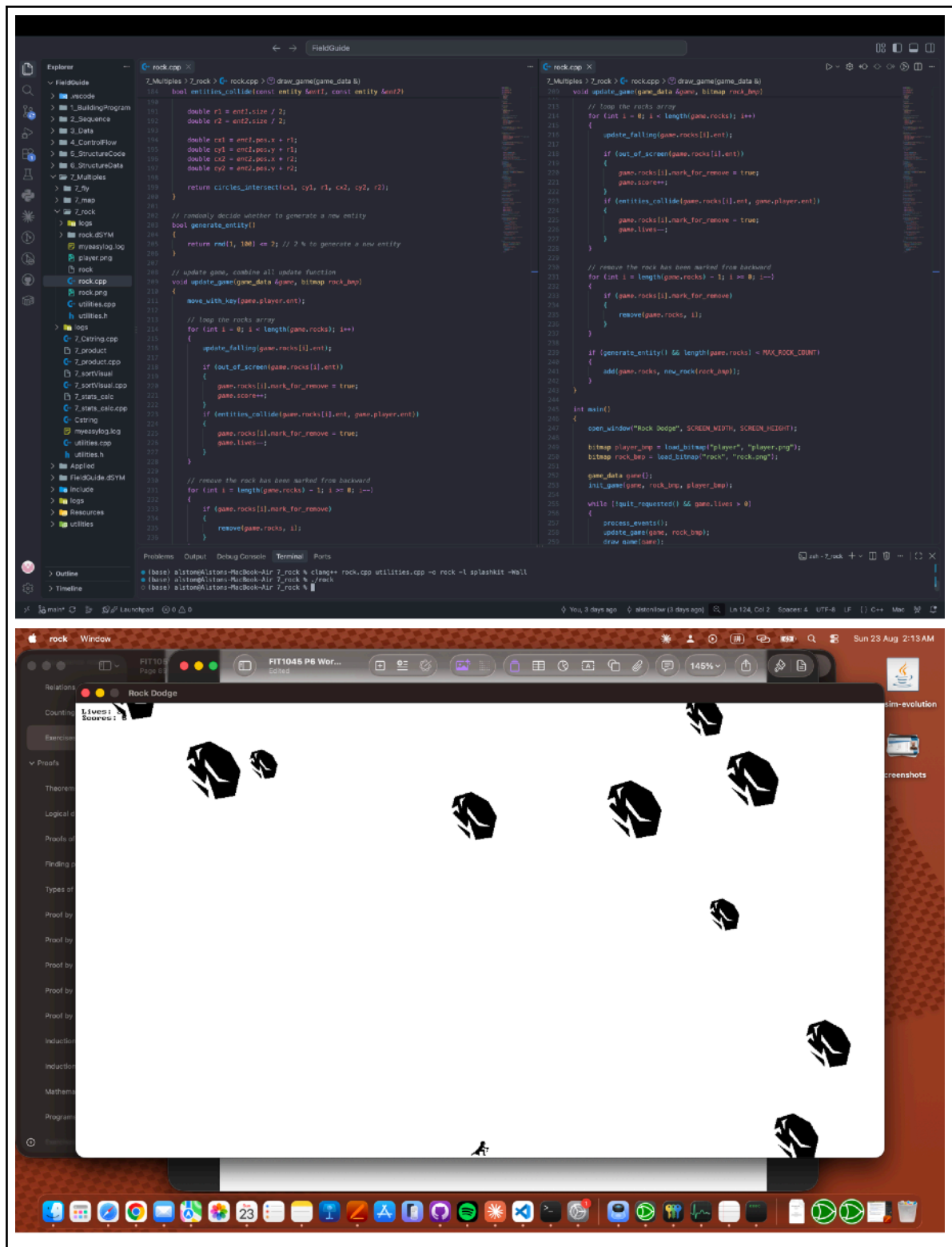
Now I want to change the circle to using sprite to present the animation. I check the splash kit and try to change my code.

When adding a bitmap, is very hassle to change all the function parameter, so I decide to add bitmap into game_data struct.

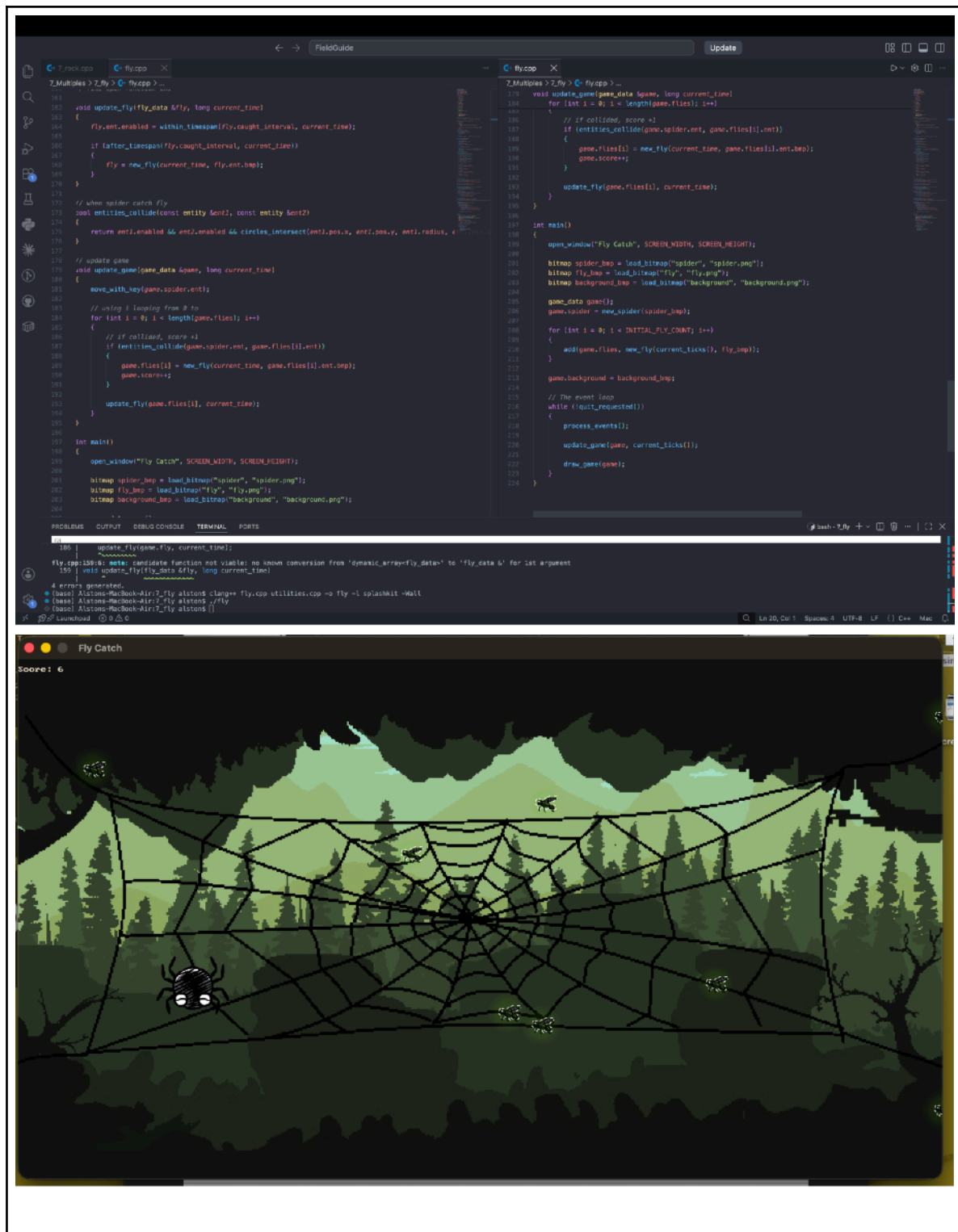
The position all get wrong after I change to bitmap and sprite, its very difficult to find the bug, I spend many time and going to have a breakdown.

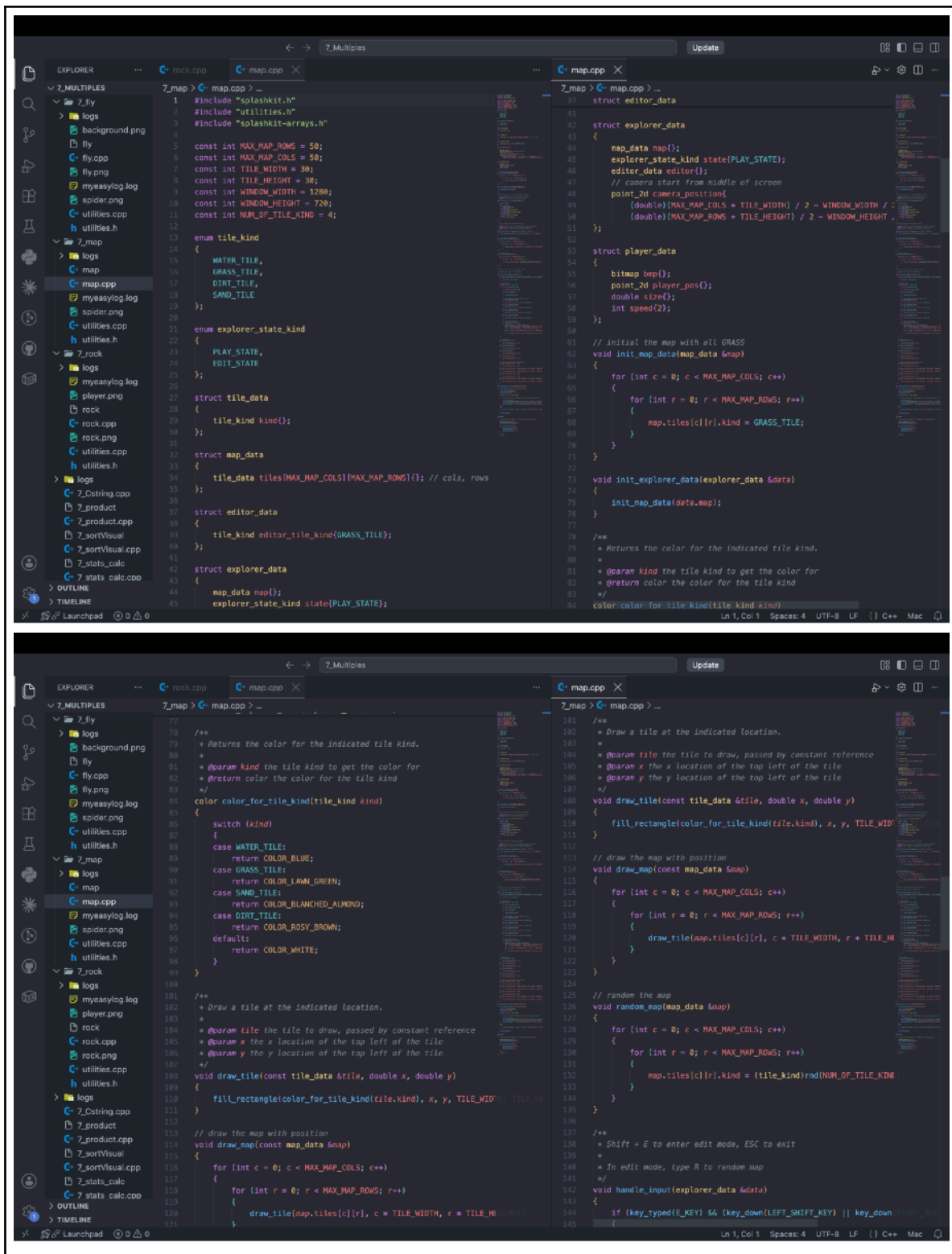
Finally, I fixed it with some calculation.





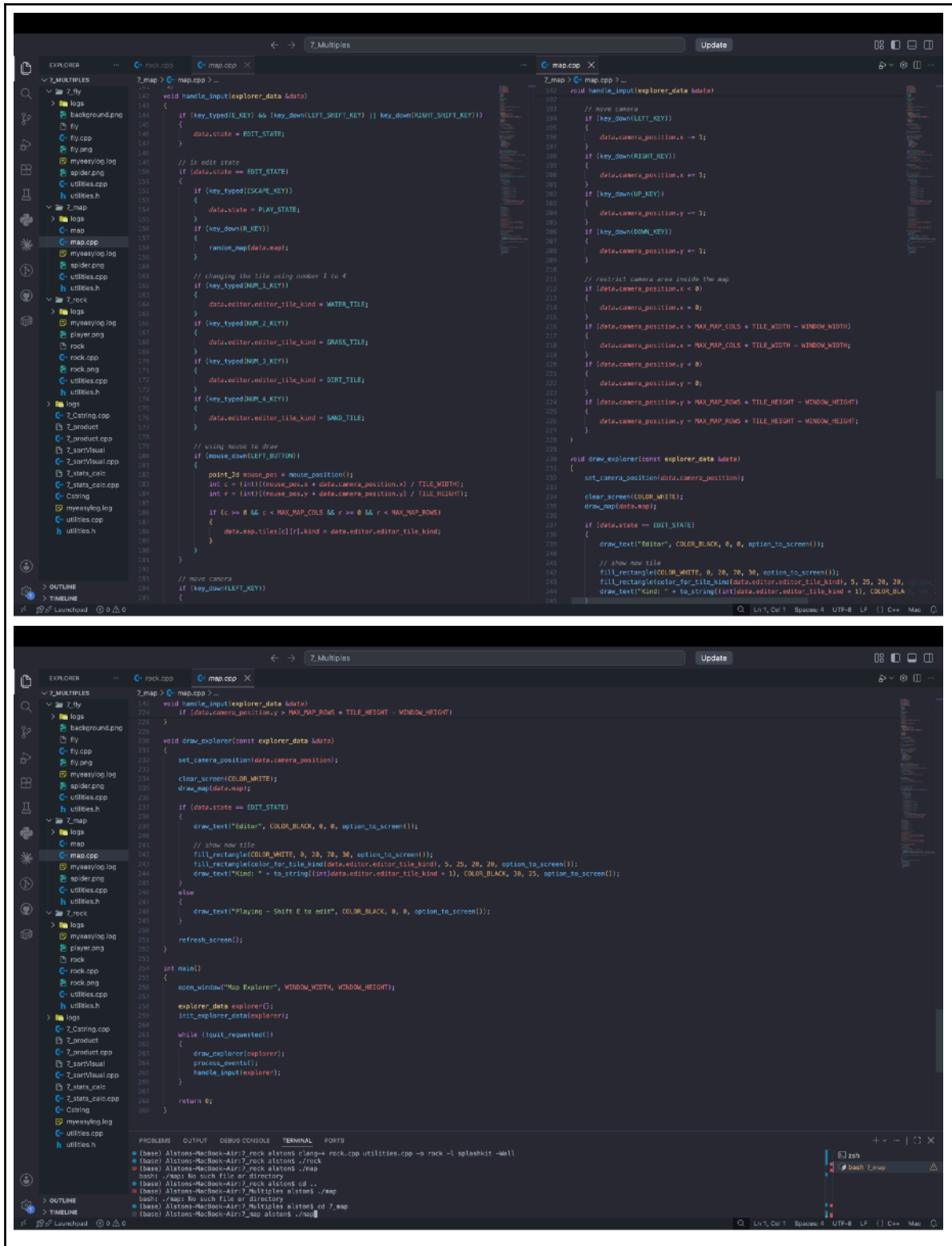
- If you want to extend your capabilities further in terms of creating larger programs, add a screenshot here showing what you were able to achieve. We recommend both the [extended fly catch tour](#), and the [map explorer tour](#). Fly catch is a quick addition to the program with structs, while map explorer will get you working with 2-dimensional arrays.





```

1  #include "splashkit.h"
2  #include "utilities.h"
3  #include "splashkit-arrays.h"
4
5  const int MAX_MAP_ROWS = 50;
6  const int MAX_MAP_COLS = 50;
7  const int TILE_WIDTH = 30;
8  const int TILE_HEIGHT = 30;
9  const int WINDOW_WIDTH = 1280;
10 const int WINDOW_HEIGHT = 720;
11 const int NUM_OF_TILE_KIND = 4;
12
13 enum tile_kind
14 {
15     WATER_TILE,
16     GRASS_TILE,
17     DIRT_TILE,
18     SAND_TILE
19 };
20
21 enum explorer_state_kind
22 {
23     PLAY_STATE,
24     EDIT_STATE
25 };
26
27 struct tile_data
28 {
29     tile_kind kind();
30 };
31
32 struct map_data
33 {
34     tile_data tiles[MAX_MAP_COLS][MAX_MAP_ROWS](); // cols, rows
35 };
36
37 struct editor_data
38 {
39     tile_kind editor_tile_kind(GRASS_TILE);
40 };
41
42 struct explorer_data
43 {
44     map_data map();
45     explorer_state_kind state(PLAY_STATE);
46
47     struct editor_data
48     {
49         map_data map();
50         explorer_state_kind state(PLAY_STATE);
51         editor_data editor();
52         // camera start from middle of screen
53         point_2d camera_position{
54             (double)(MAX_MAP_COLS * TILE_WIDTH) / 2 - WINDOW_WIDTH / 2,
55             (double)(MAX_MAP_ROWS * TILE_HEIGHT) / 2 - WINDOW_HEIGHT / 2
56         };
57     };
58
59     struct player_data
60     {
61         bitmap bmp();
62         point_2d player_pos();
63         double size();
64         int speed(2);
65     };
66
67     // initial the map with all GRASS
68     void init_map_data(map_data &map)
69     {
70         for (int c = 0; c < MAX_MAP_COLS; c++)
71         {
72             for (int r = 0; r < MAX_MAP_ROWS; r++)
73             {
74                 map.tiles[c][r].kind = GRASS_TILE;
75             }
76         }
77     }
78
79     void init_explorer_data(explorer_data &data)
80     {
81         init_map_data(data.map);
82     }
83
84     // Returns the color for the indicated tile kind.
85     // @param kind the tile kind to get the color for
86     // @return color the color for the tile kind
87     color color_for_tile_kind(tile_kind kind)
88     {
89         switch (kind)
90         {
91             case WATER_TILE:
92                 return COLOR_BLUE;
93             case GRASS_TILE:
94                 return COLOR_LAWN_GREEN;
95             case SAND_TILE:
96                 return COLOR_BLANCHED_ALMOND;
97             case DIRT_TILE:
98                 return COLOR_ROSY_BROWN;
99             default:
100                 return COLOR_WHITE;
101         }
102     }
103
104     // Draw a tile at the indicated location.
105     // @param tile the tile to draw, passed by constant reference
106     // @param x the x location of the top left of the tile
107     // @param y the y location of the top left of the tile
108     void draw_tile(const tile_data &tile, double x, double y)
109     {
110         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
111     }
112
113     // draw the map with position
114     void draw_map(const map_data &map)
115     {
116         for (int c = 0; c < MAX_MAP_COLS; c++)
117         {
118             for (int r = 0; r < MAX_MAP_ROWS; r++)
119             {
120                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
121             }
122         }
123     }
124
125     // random the map
126     void random_map(map_data &map)
127     {
128         for (int c = 0; c < MAX_MAP_COLS; c++)
129         {
130             for (int r = 0; r < MAX_MAP_ROWS; r++)
131             {
132                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
133             }
134         }
135     }
136
137     // Shift + E to enter edit mode, ESC to exit
138     // In edit mode, type A to random map
139     void handle_input(explorer_data &data)
140     {
141         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
142         {
143             data.state = EDIT_STATE;
144         }
145         else if (key_typed(A_KEY))
146         {
147             data.random_map();
148         }
149     }
150
151     // Draw a tile at the indicated location.
152     // @param tile the tile to draw, passed by constant reference
153     // @param x the x location of the top left of the tile
154     // @param y the y location of the top left of the tile
155     void draw_tile(const tile_data &tile, double x, double y)
156     {
157         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
158     }
159
160     // draw the map with position
161     void draw_map(const map_data &map)
162     {
163         for (int c = 0; c < MAX_MAP_COLS; c++)
164         {
165             for (int r = 0; r < MAX_MAP_ROWS; r++)
166             {
167                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
168             }
169         }
170     }
171
172     // random the map
173     void random_map(map_data &map)
174     {
175         for (int c = 0; c < MAX_MAP_COLS; c++)
176         {
177             for (int r = 0; r < MAX_MAP_ROWS; r++)
178             {
179                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
180             }
181         }
182     }
183
184     // Shift + E to enter edit mode, ESC to exit
185     // In edit mode, type A to random map
186     void handle_input(explorer_data &data)
187     {
188         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
189         {
190             data.state = EDIT_STATE;
191         }
192         else if (key_typed(A_KEY))
193         {
194             data.random_map();
195         }
196     }
197
198     // Draw a tile at the indicated location.
199     // @param tile the tile to draw, passed by constant reference
200     // @param x the x location of the top left of the tile
201     // @param y the y location of the top left of the tile
202     void draw_tile(const tile_data &tile, double x, double y)
203     {
204         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
205     }
206
207     // draw the map with position
208     void draw_map(const map_data &map)
209     {
210         for (int c = 0; c < MAX_MAP_COLS; c++)
211         {
212             for (int r = 0; r < MAX_MAP_ROWS; r++)
213             {
214                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
215             }
216         }
217     }
218
219     // random the map
220     void random_map(map_data &map)
221     {
222         for (int c = 0; c < MAX_MAP_COLS; c++)
223         {
224             for (int r = 0; r < MAX_MAP_ROWS; r++)
225             {
226                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
227             }
228         }
229     }
230
231     // Shift + E to enter edit mode, ESC to exit
232     // In edit mode, type A to random map
233     void handle_input(explorer_data &data)
234     {
235         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
236         {
237             data.state = EDIT_STATE;
238         }
239         else if (key_typed(A_KEY))
240         {
241             data.random_map();
242         }
243     }
244
245     // Draw a tile at the indicated location.
246     // @param tile the tile to draw, passed by constant reference
247     // @param x the x location of the top left of the tile
248     // @param y the y location of the top left of the tile
249     void draw_tile(const tile_data &tile, double x, double y)
250     {
251         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
252     }
253
254     // draw the map with position
255     void draw_map(const map_data &map)
256     {
257         for (int c = 0; c < MAX_MAP_COLS; c++)
258         {
259             for (int r = 0; r < MAX_MAP_ROWS; r++)
260             {
261                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
262             }
263         }
264     }
265
266     // random the map
267     void random_map(map_data &map)
268     {
269         for (int c = 0; c < MAX_MAP_COLS; c++)
270         {
271             for (int r = 0; r < MAX_MAP_ROWS; r++)
272             {
273                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
274             }
275         }
276     }
277
278     // Shift + E to enter edit mode, ESC to exit
279     // In edit mode, type A to random map
280     void handle_input(explorer_data &data)
281     {
282         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
283         {
284             data.state = EDIT_STATE;
285         }
286         else if (key_typed(A_KEY))
287         {
288             data.random_map();
289         }
290     }
291
292     // Draw a tile at the indicated location.
293     // @param tile the tile to draw, passed by constant reference
294     // @param x the x location of the top left of the tile
295     // @param y the y location of the top left of the tile
296     void draw_tile(const tile_data &tile, double x, double y)
297     {
298         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
299     }
300
301     // draw the map with position
302     void draw_map(const map_data &map)
303     {
304         for (int c = 0; c < MAX_MAP_COLS; c++)
305         {
306             for (int r = 0; r < MAX_MAP_ROWS; r++)
307             {
308                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
309             }
310         }
311     }
312
313     // random the map
314     void random_map(map_data &map)
315     {
316         for (int c = 0; c < MAX_MAP_COLS; c++)
317         {
318             for (int r = 0; r < MAX_MAP_ROWS; r++)
319             {
320                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
321             }
322         }
323     }
324
325     // Shift + E to enter edit mode, ESC to exit
326     // In edit mode, type A to random map
327     void handle_input(explorer_data &data)
328     {
329         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
330         {
331             data.state = EDIT_STATE;
332         }
333         else if (key_typed(A_KEY))
334         {
335             data.random_map();
336         }
337     }
338
339     // Draw a tile at the indicated location.
340     // @param tile the tile to draw, passed by constant reference
341     // @param x the x location of the top left of the tile
342     // @param y the y location of the top left of the tile
343     void draw_tile(const tile_data &tile, double x, double y)
344     {
345         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
346     }
347
348     // draw the map with position
349     void draw_map(const map_data &map)
350     {
351         for (int c = 0; c < MAX_MAP_COLS; c++)
352         {
353             for (int r = 0; r < MAX_MAP_ROWS; r++)
354             {
355                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
356             }
357         }
358     }
359
360     // random the map
361     void random_map(map_data &map)
362     {
363         for (int c = 0; c < MAX_MAP_COLS; c++)
364         {
365             for (int r = 0; r < MAX_MAP_ROWS; r++)
366             {
367                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
368             }
369         }
370     }
371
372     // Shift + E to enter edit mode, ESC to exit
373     // In edit mode, type A to random map
374     void handle_input(explorer_data &data)
375     {
376         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
377         {
378             data.state = EDIT_STATE;
379         }
380         else if (key_typed(A_KEY))
381         {
382             data.random_map();
383         }
384     }
385
386     // Draw a tile at the indicated location.
387     // @param tile the tile to draw, passed by constant reference
388     // @param x the x location of the top left of the tile
389     // @param y the y location of the top left of the tile
390     void draw_tile(const tile_data &tile, double x, double y)
391     {
392         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
393     }
394
395     // draw the map with position
396     void draw_map(const map_data &map)
397     {
398         for (int c = 0; c < MAX_MAP_COLS; c++)
399         {
400             for (int r = 0; r < MAX_MAP_ROWS; r++)
401             {
402                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
403             }
404         }
405     }
406
407     // random the map
408     void random_map(map_data &map)
409     {
410         for (int c = 0; c < MAX_MAP_COLS; c++)
411         {
412             for (int r = 0; r < MAX_MAP_ROWS; r++)
413             {
414                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
415             }
416         }
417     }
418
419     // Shift + E to enter edit mode, ESC to exit
420     // In edit mode, type A to random map
421     void handle_input(explorer_data &data)
422     {
423         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
424         {
425             data.state = EDIT_STATE;
426         }
427         else if (key_typed(A_KEY))
428         {
429             data.random_map();
430         }
431     }
432
433     // Draw a tile at the indicated location.
434     // @param tile the tile to draw, passed by constant reference
435     // @param x the x location of the top left of the tile
436     // @param y the y location of the top left of the tile
437     void draw_tile(const tile_data &tile, double x, double y)
438     {
439         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
440     }
441
442     // draw the map with position
443     void draw_map(const map_data &map)
444     {
445         for (int c = 0; c < MAX_MAP_COLS; c++)
446         {
447             for (int r = 0; r < MAX_MAP_ROWS; r++)
448             {
449                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
450             }
451         }
452     }
453
454     // random the map
455     void random_map(map_data &map)
456     {
457         for (int c = 0; c < MAX_MAP_COLS; c++)
458         {
459             for (int r = 0; r < MAX_MAP_ROWS; r++)
460             {
461                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
462             }
463         }
464     }
465
466     // Shift + E to enter edit mode, ESC to exit
467     // In edit mode, type A to random map
468     void handle_input(explorer_data &data)
469     {
470         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
471         {
472             data.state = EDIT_STATE;
473         }
474         else if (key_typed(A_KEY))
475         {
476             data.random_map();
477         }
478     }
479
480     // Draw a tile at the indicated location.
481     // @param tile the tile to draw, passed by constant reference
482     // @param x the x location of the top left of the tile
483     // @param y the y location of the top left of the tile
484     void draw_tile(const tile_data &tile, double x, double y)
485     {
486         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
487     }
488
489     // draw the map with position
490     void draw_map(const map_data &map)
491     {
492         for (int c = 0; c < MAX_MAP_COLS; c++)
493         {
494             for (int r = 0; r < MAX_MAP_ROWS; r++)
495             {
496                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
497             }
498         }
499     }
500
501     // random the map
502     void random_map(map_data &map)
503     {
504         for (int c = 0; c < MAX_MAP_COLS; c++)
505         {
506             for (int r = 0; r < MAX_MAP_ROWS; r++)
507             {
508                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
509             }
510         }
511     }
512
513     // Shift + E to enter edit mode, ESC to exit
514     // In edit mode, type A to random map
515     void handle_input(explorer_data &data)
516     {
517         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
518         {
519             data.state = EDIT_STATE;
520         }
521         else if (key_typed(A_KEY))
522         {
523             data.random_map();
524         }
525     }
526
527     // Draw a tile at the indicated location.
528     // @param tile the tile to draw, passed by constant reference
529     // @param x the x location of the top left of the tile
530     // @param y the y location of the top left of the tile
531     void draw_tile(const tile_data &tile, double x, double y)
532     {
533         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
534     }
535
536     // draw the map with position
537     void draw_map(const map_data &map)
538     {
539         for (int c = 0; c < MAX_MAP_COLS; c++)
540         {
541             for (int r = 0; r < MAX_MAP_ROWS; r++)
542             {
543                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
544             }
545         }
546     }
547
548     // random the map
549     void random_map(map_data &map)
550     {
551         for (int c = 0; c < MAX_MAP_COLS; c++)
552         {
553             for (int r = 0; r < MAX_MAP_ROWS; r++)
554             {
555                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
556             }
557         }
558     }
559
560     // Shift + E to enter edit mode, ESC to exit
561     // In edit mode, type A to random map
562     void handle_input(explorer_data &data)
563     {
564         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
565         {
566             data.state = EDIT_STATE;
567         }
568         else if (key_typed(A_KEY))
569         {
570             data.random_map();
571         }
572     }
573
574     // Draw a tile at the indicated location.
575     // @param tile the tile to draw, passed by constant reference
576     // @param x the x location of the top left of the tile
577     // @param y the y location of the top left of the tile
578     void draw_tile(const tile_data &tile, double x, double y)
579     {
580         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
581     }
582
583     // draw the map with position
584     void draw_map(const map_data &map)
585     {
586         for (int c = 0; c < MAX_MAP_COLS; c++)
587         {
588             for (int r = 0; r < MAX_MAP_ROWS; r++)
589             {
590                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
591             }
592         }
593     }
594
595     // random the map
596     void random_map(map_data &map)
597     {
598         for (int c = 0; c < MAX_MAP_COLS; c++)
599         {
600             for (int r = 0; r < MAX_MAP_ROWS; r++)
601             {
602                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
603             }
604         }
605     }
606
607     // Shift + E to enter edit mode, ESC to exit
608     // In edit mode, type A to random map
609     void handle_input(explorer_data &data)
610     {
611         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
612         {
613             data.state = EDIT_STATE;
614         }
615         else if (key_typed(A_KEY))
616         {
617             data.random_map();
618         }
619     }
620
621     // Draw a tile at the indicated location.
622     // @param tile the tile to draw, passed by constant reference
623     // @param x the x location of the top left of the tile
624     // @param y the y location of the top left of the tile
625     void draw_tile(const tile_data &tile, double x, double y)
626     {
627         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
628     }
629
630     // draw the map with position
631     void draw_map(const map_data &map)
632     {
633         for (int c = 0; c < MAX_MAP_COLS; c++)
634         {
635             for (int r = 0; r < MAX_MAP_ROWS; r++)
636             {
637                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
638             }
639         }
640     }
641
642     // random the map
643     void random_map(map_data &map)
644     {
645         for (int c = 0; c < MAX_MAP_COLS; c++)
646         {
647             for (int r = 0; r < MAX_MAP_ROWS; r++)
648             {
649                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
650             }
651         }
652     }
653
654     // Shift + E to enter edit mode, ESC to exit
655     // In edit mode, type A to random map
656     void handle_input(explorer_data &data)
657     {
658         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
659         {
660             data.state = EDIT_STATE;
661         }
662         else if (key_typed(A_KEY))
663         {
664             data.random_map();
665         }
666     }
667
668     // Draw a tile at the indicated location.
669     // @param tile the tile to draw, passed by constant reference
670     // @param x the x location of the top left of the tile
671     // @param y the y location of the top left of the tile
672     void draw_tile(const tile_data &tile, double x, double y)
673     {
674         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
675     }
676
677     // draw the map with position
678     void draw_map(const map_data &map)
679     {
680         for (int c = 0; c < MAX_MAP_COLS; c++)
681         {
682             for (int r = 0; r < MAX_MAP_ROWS; r++)
683             {
684                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
685             }
686         }
687     }
688
689     // random the map
690     void random_map(map_data &map)
691     {
692         for (int c = 0; c < MAX_MAP_COLS; c++)
693         {
694             for (int r = 0; r < MAX_MAP_ROWS; r++)
695             {
696                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
697             }
698         }
699     }
700
701     // Shift + E to enter edit mode, ESC to exit
702     // In edit mode, type A to random map
703     void handle_input(explorer_data &data)
704     {
705         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
706         {
707             data.state = EDIT_STATE;
708         }
709         else if (key_typed(A_KEY))
710         {
711             data.random_map();
712         }
713     }
714
715     // Draw a tile at the indicated location.
716     // @param tile the tile to draw, passed by constant reference
717     // @param x the x location of the top left of the tile
718     // @param y the y location of the top left of the tile
719     void draw_tile(const tile_data &tile, double x, double y)
720     {
721         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
722     }
723
724     // draw the map with position
725     void draw_map(const map_data &map)
726     {
727         for (int c = 0; c < MAX_MAP_COLS; c++)
728         {
729             for (int r = 0; r < MAX_MAP_ROWS; r++)
730             {
731                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
732             }
733         }
734     }
735
736     // random the map
737     void random_map(map_data &map)
738     {
739         for (int c = 0; c < MAX_MAP_COLS; c++)
740         {
741             for (int r = 0; r < MAX_MAP_ROWS; r++)
742             {
743                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
744             }
745         }
746     }
747
748     // Shift + E to enter edit mode, ESC to exit
749     // In edit mode, type A to random map
750     void handle_input(explorer_data &data)
751     {
752         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
753         {
754             data.state = EDIT_STATE;
755         }
756         else if (key_typed(A_KEY))
757         {
758             data.random_map();
759         }
760     }
761
762     // Draw a tile at the indicated location.
763     // @param tile the tile to draw, passed by constant reference
764     // @param x the x location of the top left of the tile
765     // @param y the y location of the top left of the tile
766     void draw_tile(const tile_data &tile, double x, double y)
767     {
768         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
769     }
770
771     // draw the map with position
772     void draw_map(const map_data &map)
773     {
774         for (int c = 0; c < MAX_MAP_COLS; c++)
775         {
776             for (int r = 0; r < MAX_MAP_ROWS; r++)
777             {
778                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
779             }
780         }
781     }
782
783     // random the map
784     void random_map(map_data &map)
785     {
786         for (int c = 0; c < MAX_MAP_COLS; c++)
787         {
788             for (int r = 0; r < MAX_MAP_ROWS; r++)
789             {
790                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
791             }
792         }
793     }
794
795     // Shift + E to enter edit mode, ESC to exit
796     // In edit mode, type A to random map
797     void handle_input(explorer_data &data)
798     {
799         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
800         {
801             data.state = EDIT_STATE;
802         }
803         else if (key_typed(A_KEY))
804         {
805             data.random_map();
806         }
807     }
808
809     // Draw a tile at the indicated location.
810     // @param tile the tile to draw, passed by constant reference
811     // @param x the x location of the top left of the tile
812     // @param y the y location of the top left of the tile
813     void draw_tile(const tile_data &tile, double x, double y)
814     {
815         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
816     }
817
818     // draw the map with position
819     void draw_map(const map_data &map)
820     {
821         for (int c = 0; c < MAX_MAP_COLS; c++)
822         {
823             for (int r = 0; r < MAX_MAP_ROWS; r++)
824             {
825                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
826             }
827         }
828     }
829
830     // random the map
831     void random_map(map_data &map)
832     {
833         for (int c = 0; c < MAX_MAP_COLS; c++)
834         {
835             for (int r = 0; r < MAX_MAP_ROWS; r++)
836             {
837                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
838             }
839         }
840     }
841
842     // Shift + E to enter edit mode, ESC to exit
843     // In edit mode, type A to random map
844     void handle_input(explorer_data &data)
845     {
846         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
847         {
848             data.state = EDIT_STATE;
849         }
850         else if (key_typed(A_KEY))
851         {
852             data.random_map();
853         }
854     }
855
856     // Draw a tile at the indicated location.
857     // @param tile the tile to draw, passed by constant reference
858     // @param x the x location of the top left of the tile
859     // @param y the y location of the top left of the tile
860     void draw_tile(const tile_data &tile, double x, double y)
861     {
862         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
863     }
864
865     // draw the map with position
866     void draw_map(const map_data &map)
867     {
868         for (int c = 0; c < MAX_MAP_COLS; c++)
869         {
870             for (int r = 0; r < MAX_MAP_ROWS; r++)
871             {
872                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
873             }
874         }
875     }
876
877     // random the map
878     void random_map(map_data &map)
879     {
880         for (int c = 0; c < MAX_MAP_COLS; c++)
881         {
882             for (int r = 0; r < MAX_MAP_ROWS; r++)
883             {
884                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
885             }
886         }
887     }
888
889     // Shift + E to enter edit mode, ESC to exit
890     // In edit mode, type A to random map
891     void handle_input(explorer_data &data)
892     {
893         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
894         {
895             data.state = EDIT_STATE;
896         }
897         else if (key_typed(A_KEY))
898         {
899             data.random_map();
900         }
901     }
902
903     // Draw a tile at the indicated location.
904     // @param tile the tile to draw, passed by constant reference
905     // @param x the x location of the top left of the tile
906     // @param y the y location of the top left of the tile
907     void draw_tile(const tile_data &tile, double x, double y)
908     {
909         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
910     }
911
912     // draw the map with position
913     void draw_map(const map_data &map)
914     {
915         for (int c = 0; c < MAX_MAP_COLS; c++)
916         {
917             for (int r = 0; r < MAX_MAP_ROWS; r++)
918             {
919                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
920             }
921         }
922     }
923
924     // random the map
925     void random_map(map_data &map)
926     {
927         for (int c = 0; c < MAX_MAP_COLS; c++)
928         {
929             for (int r = 0; r < MAX_MAP_ROWS; r++)
930             {
931                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
932             }
933         }
934     }
935
936     // Shift + E to enter edit mode, ESC to exit
937     // In edit mode, type A to random map
938     void handle_input(explorer_data &data)
939     {
940         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
941         {
942             data.state = EDIT_STATE;
943         }
944         else if (key_typed(A_KEY))
945         {
946             data.random_map();
947         }
948     }
949
950     // Draw a tile at the indicated location.
951     // @param tile the tile to draw, passed by constant reference
952     // @param x the x location of the top left of the tile
953     // @param y the y location of the top left of the tile
954     void draw_tile(const tile_data &tile, double x, double y)
955     {
956         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
957     }
958
959     // draw the map with position
960     void draw_map(const map_data &map)
961     {
962         for (int c = 0; c < MAX_MAP_COLS; c++)
963         {
964             for (int r = 0; r < MAX_MAP_ROWS; r++)
965             {
966                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
967             }
968         }
969     }
970
971     // random the map
972     void random_map(map_data &map)
973     {
974         for (int c = 0; c < MAX_MAP_COLS; c++)
975         {
976             for (int r = 0; r < MAX_MAP_ROWS; r++)
977             {
978                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
979             }
980         }
981     }
982
983     // Shift + E to enter edit mode, ESC to exit
984     // In edit mode, type A to random map
985     void handle_input(explorer_data &data)
986     {
987         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
988         {
989             data.state = EDIT_STATE;
990         }
991         else if (key_typed(A_KEY))
992         {
993             data.random_map();
994         }
995     }
996
997     // Draw a tile at the indicated location.
998     // @param tile the tile to draw, passed by constant reference
999     // @param x the x location of the top left of the tile
1000    // @param y the y location of the top left of the tile
1001    void draw_tile(const tile_data &tile, double x, double y)
1002    {
1003        fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
1004    }
1005
1006    // draw the map with position
1007    void draw_map(const map_data &map)
1008    {
1009        for (int c = 0; c < MAX_MAP_COLS; c++)
1010        {
1011            for (int r = 0; r < MAX_MAP_ROWS; r++)
1012            {
1013                draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
1014            }
1015        }
1016    }
1017
1018    // random the map
1019    void random_map(map_data &map)
1020    {
1021        for (int c = 0; c < MAX_MAP_COLS; c++)
1022        {
1023            for (int r = 0; r < MAX_MAP_ROWS; r++)
1024            {
1025                map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
1026            }
1027        }
1028    }
1029
1030    // Shift + E to enter edit mode, ESC to exit
1031    // In edit mode, type A to random map
1032    void handle_input(explorer_data &data)
1033    {
1034        if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
1035        {
1036            data.state = EDIT_STATE;
1037        }
1038        else if (key_typed(A_KEY))
1039        {
1040            data.random_map();
1041        }
1042    }
1043
1044    // Draw a tile at the indicated location.
1045    // @param tile the tile to draw, passed by constant reference
1046    // @param x the x location of the top left of the tile
1047    // @param
```



```

// map.h
void handle_input(explorer_data &data);
void draw_explorer(const explorer_data &data);

// map.cpp
void handle_input(explorer_data &data)
{
    if (key_pressed(L_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
    {
        data.state = EDIT_STATE;
    }
    // in edit state
    if (data.state == EDIT_STATE)
    {
        if (key_pressed(ESCAPE_KEY))
        {
            data.state = PLAY_STATE;
        }
        if (key_down(R_KEY))
        {
            random_map(data.map);
        }
        // changing the tile using number 1 to 4
        if (key_pressed(MAP_1_KEY))
        {
            data.editor.editor_tile_kind = WATER_TILE;
        }
        if (key_pressed(MAP_2_KEY))
        {
            data.editor.editor_tile_kind = GRASS_TILE;
        }
        if (key_pressed(MAP_3_KEY))
        {
            data.editor.editor_tile_kind = DIRT_TILE;
        }
        if (key_pressed(MAP_4_KEY))
        {
            data.editor.editor_tile_kind = SAND_TILE;
        }
        // using mouse to draw
        if (mouse_down(LEFT_BUTTON))
        {
            point_2d mouse_pos = mouse_position();
            int c = (int)((mouse_pos.x + data.camera.position.x) / TILE_WIDTH);
            int r = (int)((mouse_pos.y + data.camera.position.y) / TILE_HEIGHT);
            if (c >= 0 && c < MAX_MAP_COLS && r >= 0 && r < MAX_MAP_ROWS)
            {
                data.map.tiles[c][r].kind = data.editor.editor_tile_kind;
            }
        }
    }
    // move camera
    if (key_down(LEFT_KEY))
    {
        data.camera.position.x -= 1;
    }
    if (key_down(RIGHT_KEY))
    {
        data.camera.position.x += 1;
    }
    if (key_down(UP_KEY))
    {
        data.camera.position.y -= 1;
    }
    if (key_down(DOWN_KEY))
    {
        data.camera.position.y += 1;
    }
    // restrict camera area inside the map
    if (data.camera.position.x < 0)
    {
        data.camera.position.x = 0;
    }
    if (data.camera.position.x > MAX_MAP_COLS + TILE_WIDTH - WINDOW_WIDTH)
    {
        data.camera.position.x = MAX_MAP_COLS + TILE_WIDTH - WINDOW_WIDTH;
    }
    if (data.camera.position.y < 0)
    {
        data.camera.position.y = 0;
    }
    if (data.camera.position.y > MAX_MAP_ROWS + TILE_HEIGHT - WINDOW_HEIGHT)
    {
        data.camera.position.y = MAX_MAP_ROWS + TILE_HEIGHT - WINDOW_HEIGHT;
    }
}

void draw_explorer(const explorer_data &data)
{
    set_camera_position(data.camera.position);
    clear_screen(COLOR_WHITE);
    draw_map(data.map);
    if (data.state == EDIT_STATE)
    {
        draw_text("Editor", COLOR_BLACK, 0, 0, option_to_screen());
        // show new tile
        fill_rectangle(COLOR_WHITE, 0, 20, 30, 30, option_to_screen());
        fill_rectangle(COLOR_WHITE, 5, 25, 20, 20, option_to_screen());
        draw_text("kind = " + to_string((int)data.editor.editor_tile_kind + 1), COLOR_BLACK, 30, 25, option_to_screen());
    }
    else
    {
        draw_text("Playing - Shift E to edit", COLOR_BLACK, 0, 0, option_to_screen());
    }
    refresh_screen();
}

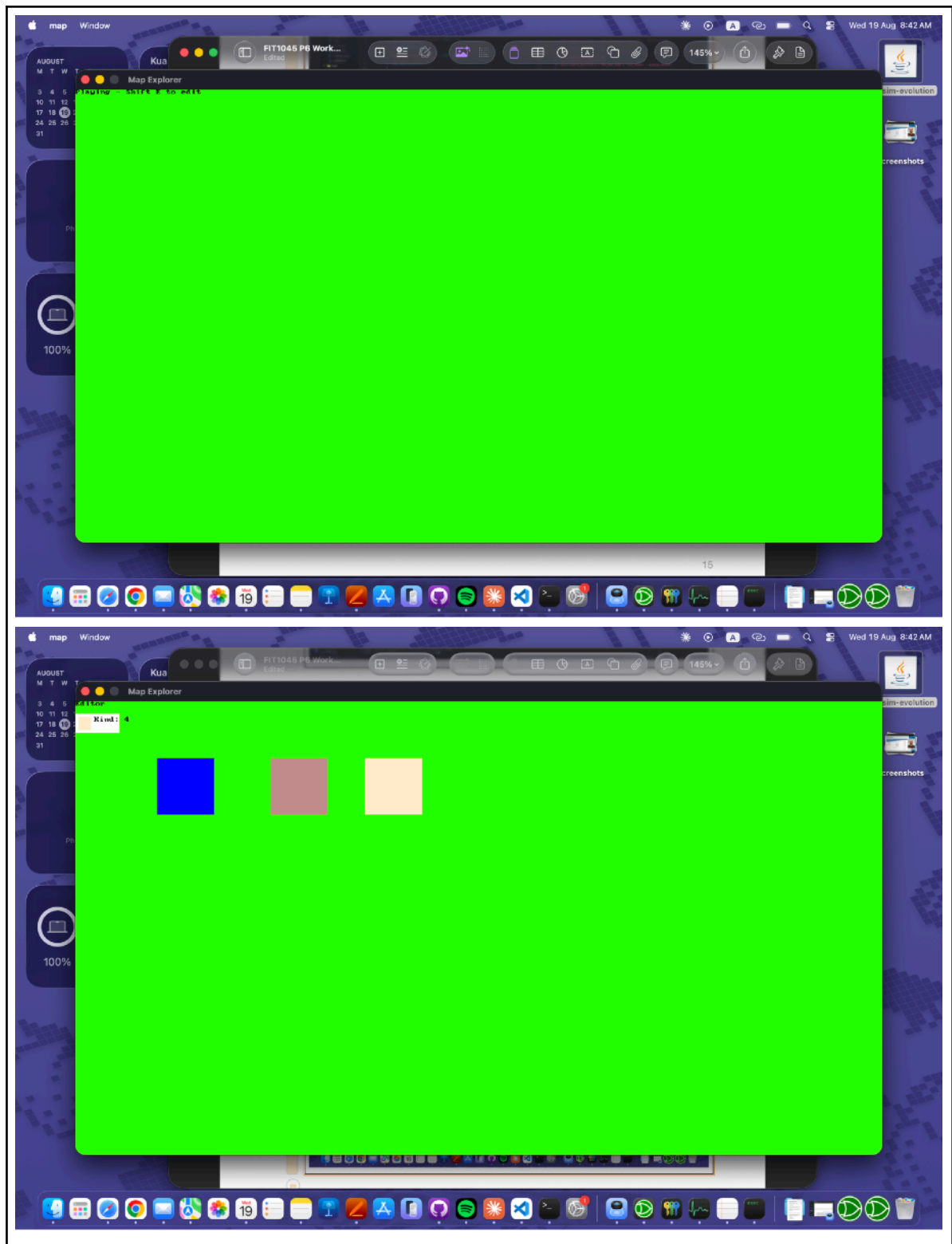
int main()
{
    open_window("Map Explorer", WINDOW_WIDTH, WINDOW_HEIGHT);
    explorer_data explorer;
    init_explorer_data(explorer);
    while (!quit_requested())
    {
        draw_explorer(explorer);
        process_events();
        handle_input(explorer);
    }
    return 0;
}

```

```

(base) Alstons-MacBook-Air:~$ rock -c alstons -l rock -c utilities.cpp -o rock -l splashkit -Wall
(base) Alstons-MacBook-Air:~$ rock alstons ./rock
(base) Alstons-MacBook-Air:~$ rock alstons ./map
(base) Alstons-MacBook-Air:~$ rock alstons ./map
(base) Alstons-MacBook-Air:~$ rock alstons ./map
(base) Alstons-MacBook-Air:~$ rock alstons ./map

```

**Reflect:**

- What was the most interesting thing you learnt through this task, and why?

> I learned about how to build a complex and huge project step by step. First need to build a frame about this project, separate a big problem to many small problem. Then for each small problem, we need to think how to solve it, can use function, procedure or otherwise to solve it. The combine all the solution to become a full version program do complete the initial goal.

- How are arrays and loops connected?

> Arrays in c++ is very inconvenient to do some operation. Using loop we can do some operation like reading, adding, removing or use it. Every time using loop in array, need to using int i to looping arrays[i].

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I use 32 hours for this task. I many time to doing map explorer and rock dodge. When I finish the initial version of rock dodge using circle, I want to changing it to sprite, I face many problem so then I use bitmap to replace sprite but it also cannot work, the position of rock and player get wrong and I cannot fix it. I debug very long time to fix this.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude to guide me doing the map explorer to add some function.
In the Rock dodge game, I ask about how to remove the marked rock from the array and also help me to check the position problem.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:
- Engage with feedback and discussion to get this signed off.